

Asincronía en JavaScript. Callbacks. Promises. Async/Await

Autoría: Rubén Rodríguez Paz

El encargo y la creación de este recurso de aprendizaje UOC han sido coordinados por el profesor: Javier Luis

Cánovas Izquierdo

PID_00293444

Primera edición: febrero 2023

1. Introducción a la asincronía en JavaScript

- 1.1. Introducción
- 1.2. Asincronía paso a paso
- 1.3. El *event loop*
- 1.4. Ejercicios

2. Callbacks

- 2.1. Introducción
- 2.2. *Callbacks* anidados y *callback hell*
- 2.3. Problemas de fiabilidad
- 2.4. Ejercicios

3. Promesas

- 3.1. Introducción
- 3.2. Qué es una promesa
- 3.3. `Then()`
- 3.4. La API de *promise*
- 3.5. Ejercicios

4. Generadores

- 4.1. Introducción
- 4.2. Generadores y *promises*
- 4.3. *Async/Await*
- 4.4. Ejercicios

5. Soluciones de los ejercicios

Bibliografía

1. Introducción a la asincronía en JavaScript

1.1. Introducción

Desde los orígenes de la web, JavaScript ha sido una de las tecnologías fundamentales encargadas de ofrecer una experiencia interactiva al contenido que se consume. Desde efectos *pop-up* en *banners* de publicidad hasta animaciones (ahora anticuadas) del cursor al moverse sobre el navegador.

Con el tiempo, el lenguaje ha ido evolucionando, y ha pasado de ser un lenguaje auxiliar de *scripting* a algo fundamental en la operativa actual de consumo de contenido mediante las API y arquitecturas basadas en microservicios. En estos modelos, uno de los conceptos fundamentales es la gestión del programa y sus datos a lo largo del tiempo.

En un modelo de programación síncrono, todas las acciones se desarrollan de un modo lineal. Cuando se ejecuta una función, la ejecución del programa espera hasta que la función en ejecución retorna un valor, independientemente del tiempo que le lleve.

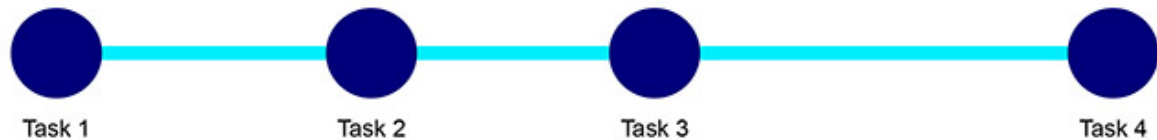


Figura 1. Representación de ejemplo del flujo de ejecución de una aplicación síncrona

En un modelo de programación asíncrono, varias acciones se pueden desarrollar en paralelo a la vez. Al igual que en la programación síncrona la ejecución del programa principal espera, en el caso de la programación asíncrona puede haber múltiples funciones en ejecución con flujos que se entrelazan y retornos de función que modifican el comportamiento de la aplicación principal en el tiempo.

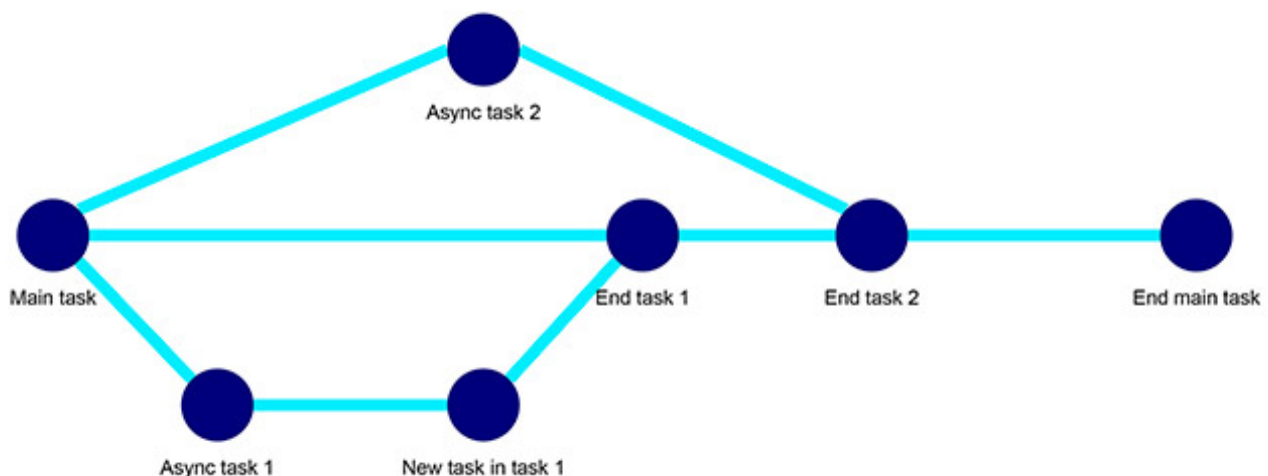


Figura 2. Representación de ejemplo del flujo de una aplicación síncrona

1. Introducción a la asincronía en JavaScript

1.2. Asincronía paso a paso

Para ejemplificar un caso básico de gestión de asincronía vamos a analizar paso a paso una porción de código que realice una llamada a un método asíncrono.

Digamos que hemos desarrollado un método o usamos una librería que realiza llamadas mediante AJAX:

```
var data = ajaxMethod("http://url.com");  
console.log(data);
```

Nota

AJAX hace mención a JavaScript y XML asíncrono (*asynchronous JavaScript and XML*). Es una técnica para realizar aplicaciones asíncronas en las que el cliente o navegador realiza una petición en segundo plano, y una vez recibida la respuesta se realizan las operaciones pertinentes o se modifica el comportamiento del cliente. Este en el siguiente documento realiza una descripción más detallada de las técnicas utilizadas y ejemplos: <https://developer.mozilla.org/es/docs/Web/Guide/AJAX>.

Debido a que no se sabe cuándo va a terminar la llamada AJAX, y el código se ejecuta de modo lineal, lo más probable es que no se muestren los datos de la variable `data` al hacer `console.log()`, aunque podría darse algún caso en que sí. Esta incertidumbre en el resultado no es algo deseado. Una posible solución es ejecutar la función con un `timeout` y esperar que devuelva los datos de la petición:

```
var data = ajaxMethod("http://url.com");  
setTimeout(function() {  
    console.log(data);  
}, 3000);
```

Sin embargo, en este caso tenemos un problema similar al anterior, ya que aunque estamos dando un margen de 3 segundos para que se realice la llamada AJAX, no tenemos la certeza de que termine a tiempo. En el caso en el que lo haga, tenemos que esperar a que terminen los 3 segundos para continuar con la ejecución, con lo que tenemos un programa altamente ineficiente.

Entendemos que la necesidad que tenemos en nuestro código es que la llamada a `ajaxMethod` bloquee la ejecución del programa principal hasta que tengamos una respuesta del método; en ese momento sería cuando tendríamos un valor de `data` válido.

Sin embargo, este no es el comportamiento estándar de AJAX en JavaScript, por lo que tenemos que buscar modos alternativos para realizar este control.

El método más sencillo, consistente en esperar hasta que la función termine, es mediante el uso de una función `callback`. De esta manera, la función `callback` se llamará una vez termine la petición asíncrona. El siguiente ejemplo ilustra esta solución:

```
ajaxMethod("http://url.com", function callbackFunction(data) {  
    console.log(data);  
});
```

Aunque efectivo, este modo de gestionar peticiones asíncronas puede llevar a una serie de problemas, como veremos más adelante.

1. Introducción a la asincronía en JavaScript

1.3. El event loop

Una de las razones por las que el comportamiento de los fragmentos de código sea como los descritos previamente se debe al *event loop*, uno de los fundamentos principales sobre los que se sustenta JavaScript como lenguaje.

Según la definición de MDN de *event loop*:

«El *loop* de eventos obtiene su nombre por la forma en que es usualmente implementado, la cual generalmente se parece a:

```
while (queue.waitForMessage()) {  
  queue.processNextMessage();  
}
```

`queue.waitForMessage` espera de manera síncrona a que llegue un mensaje si no hay ninguno actualmente».

Enlace recomendado

Podéis consultar la definición de MDN de *event loop* en el siguiente enlace:

<https://developer.mozilla.org/es/docs/Web/JavaScript/EventLoop>.

El *event loop* tiene una serie de propiedades que es importante tener en cuenta.

- **Ejecutar hasta completar.** Cada mensaje, o cada bloque en el caso de JavaScript, se procesa completamente antes de continuar con el siguiente, esto ofrece ventajas de cara a predecir el comportamiento de aplicaciones síncronas. También ofrece alguna desventaja básica, como que si un bloque dura demasiado tiempo en ejecutarse, la aplicación queda bloqueada.
- **Mensajes y eventos.** Los navegadores web añaden mensajes a la cola de eventos cada vez que un evento ocurre, que posteriormente pueden ser capturados o no por otros fragmentos de código. Esto, entre otras cosas, condiciona el uso de métodos de gestión del tiempo de ejecución como `setTimeout`, ya que el método no se salta su «turno» en la cola del *event loop*. Por tanto, deberá esperar a que se ejecuten todos los mensajes y sean procesados. Esta es la razón por la que al fijar un `timeout`, el parámetro de tiempo es un tiempo mínimo esperado, pero en ningún caso supone una garantía.
- **Cero retraso.** Tomando el ejemplo previo de uso del método `setTimeout`, al llamar al método `setTimeout`, y aunque se fije como valor de retraso 0 milisegundos, no significa que su contenido se vaya a ejecutar inmediatamente, ya que debe esperar a ejecutar los bloques pendientes que queden en la pila de llamadas.

Veamos el funcionamiento del *event loop* con un ejemplo:

```
function eventLoopTest() {  
  console.log('1');  
  setTimeout(  
    function showNumber() { console.log('2'); },  
    0);  
  console.log('3');  
}  
eventLoopTest();
```

¿Qué se mostrará por pantalla y en qué orden? Al contrario del pensamiento inicial, **se mostrará 1, 3, 2.**

Primero, la llamada a la función `eventLoopTest` se meterá en la pila de llamadas y comenzará a ejecutarse. La primera línea es un `console.log`, por lo que se mostrará por pantalla, en este caso «1». La siguiente instrucción es la llamada al método `setTimeout()` con una función como retorno. Aun teniendo un retraso de 0 milisegundos, se trata de un bloque nuevo, por lo que deberá encolarse para ser ejecutado posteriormente.

Finalmente, se ejecuta `console.log('3')`, y se finaliza la ejecución de `eventLoopTest`, sacándolo de la pila de llamadas.

El siguiente bloque que habría que ejecutar en el *event loop* sería el bloque de la función `setTimeout()`, tal y como se ha comentado anteriormente. Así, para terminar la ejecución, se ejecuta el contenido de la función de retorno de `setTimeout()`, que muestra por pantalla «2».

1. Introducción a la asincronía en JavaScript

1.4. Ejercicios

Ejercicio 1

Dado el siguiente código, ¿cuál sería su salida por consola? ¿Por qué?

```
setTimeout( function() { console.log("Timeout"); }, 1000 );  
function randomFunction() { console.log("Function"); }  
console.log("Main Block");
```

Ejercicio 2

Modifica la función previa para que se ejecuten todos los `console.log()` .

¿Habría otros modos de instanciar `randomFunction()` ?

Ejercicio 3

Dado el siguiente código, ¿cuál sería su salida por consola? ¿Por qué? Ten en cuenta que son funciones flecha.

```
const first = () => console.log('First');  
const second = () => setTimeout(() => console.log('Second'));  
const third = () => console.log('Third');
```

Ejercicio 4

Dado el siguiente código, ¿cuál sería su salida por consola? ¿Por qué?

```
for ( var i = 0; i < 3; i++) {  
    setTimeout( function() { console.log(i); }, i * 1000 );  
}
```

Ejercicio 5

Dado el siguiente código, ¿cuál sería su salida por consola? ¿Por qué?

```
function first() {  
    console.log(1)  
}  
  
function second() {  
    setTimeout(() => {  
        console.log(2)  
    }, 0)  
}  
  
function third() {  
    console.log(3)  
}  
first()  
second()  
third()
```

Ejercicio 6

Dado el siguiente código, ¿cuál sería su salida por consola? ¿Por qué?

```
( function() {  
    console.log(10);  
    setTimeout( function() {console.log(20)}, 1000);  
    setTimeout( function() {console.log(30)}, 0);  
    console.log(40);  
} )();
```

2. Callbacks

2.1. Introducción

Como hemos visto, el *event loop* hace que el comportamiento de las aplicaciones pueda ser predecible. Sin embargo, al entrar en juego la asincronía, su comportamiento se puede volver tramposo.

El primer intento que podemos realizar para gestionar la asincronía y realizar acciones una vez la llamada asíncrona termine es mediante el uso de funciones que tomen un argumento extra, una función `callback`. El objetivo de esta función es que sea llamada cuando la llamada asíncrona termine, como el ejemplo que sigue.

```
ajaxMethod("http://url.com", function callbackFunction(data) {  
    console.log(data);  
});
```

Un *callback* es simplemente una función que se debe ejecutar una vez que se cumpla una situación o circunstancia. En el caso de las peticiones asíncronas, el `callback` se ejecuta cuando los datos llegan devueltos desde el servidor. Este patrón de programación en JavaScript es una de las formas más básicas de gestionar la asincronía.

Las funciones que aceptan otras funciones como argumentos son llamadas funciones de orden superior (*high-order function*), y son las que se encargan de determinar cuándo se ejecuta la función `callback`. Es la combinación de estas dos funciones (la función de orden superior y la función `callback`) la que nos permite ampliar la funcionalidad para el soporte de la asincronía y control del orden de ejecución en JavaScript. Veámoslo con un ejemplo un poco más completo:

```
function createQuote(quote, callback) {  
    const myQuote = "That's one small step for man, " + quote;  
    callback(myQuote);  
}  
  
function printQuote(quote) {  
    console.log(quote);  
}  
  
createQuote("one giant leap for mankind.", printQuote);
```

En el ejemplo previo, `createQuote` es la función de orden superior, que acepta dos argumentos. El segundo argumento es el `callback` (independientemente del nombre del argumento, que en este caso coincide).

Es importante mencionar que cuando llamamos a la función `createQuote` no estamos añadiendo paréntesis a la función `callback(printQuote)` al pasarla como argumento. La razón de hacer esto es que no queremos ejecutar dicha función inmediatamente. Queremos que se ejecute cuando toque en la función de orden superior, pero es necesario pasar la definición de la misma para que pueda ser gestionada.

Además, es importante asegurarse de que se proporcionan los argumentos en el caso de que la función `callback` los requiera. En el ejemplo anterior estaríamos hablando de `callback(myQuote)`, ya que la función `printQuote` requiere un argumento.

2. Callbacks

2.2. Callbacks anidados y callback hell

Tengamos en cuenta la siguiente porción de código en la que se encadenan múltiples llamadas asíncronas:

```
listen("click", function handler(event) {  
    setTimeout( function request() {  
        ajaxFunction("http://url.com", function response(data) {  
            if (data == "case 1") {  
                handler();  
            }  
            else if (data == "case 2") {  
                request();  
            }  
        } );  
    }, 5000) ;  
} );
```

Como se puede observar, al ir encadenando funciones asíncronas, el código se hace cada vez menos reconocible y realizar un análisis razonado de qué está pasando o qué porción de código se está ejecutando es mucho más complicado. Esto es conocido como el *callback hell* y, en ocasiones, pirámide de la perdición (*pyramid of doom*), por la forma que va tomando el código debido a la indentación.

En cualquier caso, el *callback hell* no tiene nada que ver con la indentación. Es un problema mucho mayor. Para ilustrarlo de manera explícita, vamos a separar el código previo en bloques.

Bloque 1:

```
listen("click", function handler(event) {  
});
```

Bloque 2:

```
setTimeout(function request() {  
}, 5000);
```

Bloque 3:

```
ajaxFunction("http://url.com",  
function response(data) {  
});
```

Bloque 4:

```
if (data == "case 1") {  
}  
else if (data == "case 2") {  
}
```

Este código de ejemplo tiene una ventaja respecto a lo que podría encontrarse en situaciones reales, ya que el contenido de las llamadas es mínimo y se puede ver tanto la anidación como los *callbacks* de un solo vistazo. Cualquier programa convencional que use *callbacks* cuenta con el suficiente número de líneas, métodos y bloques como para hacer el seguimiento mucho más difícil. Sin embargo, hay un problema mucho más complejo. Para ello vamos a poner otro ejemplo:

```
functionA( function() {
    functionB();
    functionC( function() {
        functionD();
    } )
    functionE();
} );

functionF();
```

A pesar de lo que pueda parecer a primera vista, el orden de ejecución del código (que no de resolución y retorno de los métodos) es el siguiente:

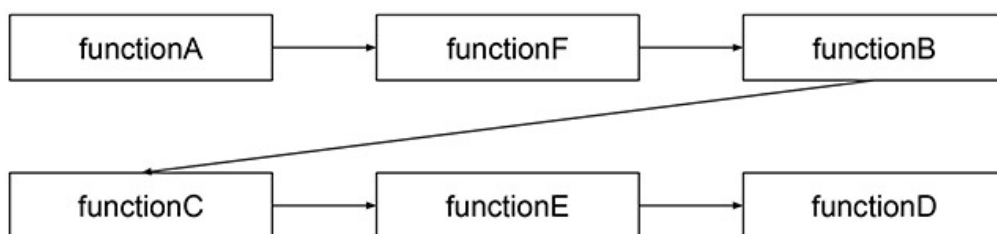


Figura 3. Representación del orden de ejecución del código previo

Como se puede ver, su análisis no es tan natural ni lineal como se esperaba en primera instancia, y si se mezclaran llamadas síncronas con llamadas asíncronas, navegar por el código sería un gran problema de comprensión, de ahí el uso de la denominación *callback hell*.

2. Callbacks

2.3. Problemas de fiabilidad

El problema con el razonamiento secuencial y la gestión de asincronía mediante *callbacks* es solo uno de los problemas que tiene el hecho de desarrollar de este modo.

Poniendo como ejemplo una sencilla función con callback:

```
// A
ajaxFunction("http://url.com", function response(data) {
  //C
});
//B
```

A y **B** se ejecutan instantáneamente, ya que es parte del programa principal, mientras que **C** ocurre más tarde, bajo el control de `ajaxFunction`. Siendo optimistas, este método no tendría por qué fallar. Pero en muchos casos este tipo de método podría provenir de una librería de terceros, código desarrollado por otras personas u otros métodos de los cuales no podemos controlar su funcionamiento. Este fenómeno es llamado **inversión de control**, y ocurre cuando se delega la ejecución de un bloque de código a la ejecución de otro bloque de código.

2. Callbacks

2.4. Ejercicios

Ejercicio 7

Modifica el siguiente código para que cumpla con los siguientes requisitos:

- Modificar la función `greet`, para que reciba también un método como parámetro. Este método debe ejecutarse al terminar la ejecución de la función.
- Ejecutar la función `greet` pasando como parámetros el nombre *Pepito* y la función `farewell`.

```
function greet(name) {  
  console.log('Hello Mr.' + ' ' + name);  
}  
  
function farewell() {  
  console.log('Goodbye Mr. Jose');  
}
```

Ejercicio 8

Modifica el siguiente código para que cumpla con los siguientes requisitos:

- Modificar la función `greet`, para que reciba también un método como parámetro. Este método debe ejecutarse al terminar la ejecución de la función, pasando como parámetro `name`.
- Ejecutar la función `greet` pasando como parámetros el nombre *Pepito* y la función `farewell`, con un temporizador de 3 segundos.

```
function greet(name) {  
  console.log('Hello Mr.' + ' ' + name);  
}  
  
function farewell(name) {  
  console.log('Goodbye Mr.' + ' ' + name);  
}
```

Ejercicio 9

Modifica el siguiente código para que cumpla con los siguientes requisitos:

- Modificar la función `buildCar`, para que reciba también un método como parámetro. Este método debe ejecutarse al terminar la ejecución de la función, pasando como parámetro `name`.
- Ejecutar la función `buildCar` pasando como parámetros el nombre *Ford Focus* y la función `defineCar`.

```
function buildCar(name) {  
  setTimeout(() => {  
    console.log('Building a' + ' ' + name);  
  }, 1000);  
}  
  
function defineCar(name) {  
  console.log('This car is a' + ' ' + name);  
}
```

Ejercicio 10

¿Cuál sería la salida por pantalla del siguiente fragmento de código? Intenta deducirlo sin tener que ejecutarlo. Describe brevemente su comportamiento.

```
function hellishFunction() {
  setTimeout(() => {
    console.log(1)
    setTimeout(() => {
      console.log(2)
      setTimeout(() => {
        console.log(3)
      }, 500)
    }, 2000)
  }, 1000)
}
```

Ejercicio 11

¿Cuál sería la salida por pantalla del siguiente fragmento de código? Intenta deducirlo sin tener que ejecutarlo. Describe brevemente su comportamiento.

```
const visited = []

function walk() {

  setTimeout(function() {
    visited.push('London');

    setTimeout(function() {
      visited.push('Milan');

      setTimeout(function() {
        visited.push('Madrid');

        setTimeout(function() {
          visited.push('Barcelona');
          console.log(visited.toString())
        }, 1000);
      }, 1000);
    }, 1000);
  }, 1000);
}

walk();
```

Ejercicio 12

¿Cuál sería la salida por pantalla del siguiente fragmento de código? Intenta deducirlo sin tener que ejecutarlo. Describe brevemente su comportamiento.

```
const mainFunction = (callback) => {
  setTimeout(() => {
    callback([10, 3, 7]);
  }, 1000);
}
```

```
    }, 3000)
  }

  const addFunction = (array) => {
    let sum = 0;
    for (let i of array) {
      sum += i;
    }
    console.log(sum);
  }

  mainFunction(add);
```

3. Promesas

3.1. Introducción

Como se ha comentado previamente, cuando se trata de gestionar la asincronía mediante *callbacks*, nos podemos encontrar con dos problemas a la hora de tratarlos:

- **Falta de orden o secuencialidad de las llamadas**, cuyo resumen más claro se puede ver en casos de *callback hell*.
- **Fiabilidad de las llamadas con *callback***, que por su construcción derivan en problemas de inversión de control

```
ajaxFunction( "http://url.com", function responseMethod(data) {  
  ...something  
});
```

En este ejemplo, cuando concluya `ajaxFunction`, se ejecutará `responseMethod`.

Esto es lo que pasará, cuando se termine de ejecutar esta función.

Pero ¿y si pudiéramos modificar la forma de trabajar y en lugar de dirigir nuestra atención a otra parte del código con el `callback` pudiéramos continuar leyendo y decidir qué hará nuestro programa después?

Esto, y los problemas de fiabilidad de llamadas, motivaron la aparición de un nuevo paradigma en la programación en JavaScript: las ***promises*** ('promesas').

Debido a la utilidad de este paradigma y a la urgente necesidad por parte de los desarrolladores de abordar los problemas comentados anteriormente, las promesas han sido ampliamente adoptadas por la comunidad de desarrolladores, ya que soluciona de modo inmediato el *callback hell* haciendo que el desarrollo de aplicaciones con grandes cargas de asincronía sea más amigable.

3. Promesas

3.2. Qué es una promesa

Según la definición de MDN de *promise*:



«Una promesa es un objeto que representa la terminación o el fracaso de una operación asíncrona».

Enlace recomendado

Podéis consultar la definición de MDN de *promise* en el siguiente enlace:

https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Global_Objects/Promise.

Aunque la definición es bastante abstracta, ya habla del estado y fiabilidad de la operación asíncrona, por lo que pretende solucionar los problemas previos de asincronía con *callbacks*. Antes de definir a nivel técnico qué es una *promesa* vamos a ejemplificar su comportamiento con una analogía:

Imaginemos que nos encontramos en el mostrador de una pizzería de tipo «comida rápida» y pedimos una pizza de jamón y queso. Le entregamos el dinero que cuesta, en este caso 15 €. Al pedirle al dependiente la pizza y pagar, hemos hecho una petición con un valor de vuelta (en este caso una pizza de jamón y queso). Hemos comenzado una transacción.

Pero, a menudo, la pizza no está disponible inmediatamente, por lo que tenemos que esperar. En lugar de recibir la pizza al momento, cosa que no es posible, el dependiente nos da un ticket con un número de pedido. Este número de pedido asegura (promesa) que en cierto momento recibiré mi pizza.

Mientras espero, podemos realizar otras acciones, como buscar un sitio para sentarnos o coger servilletas. Este tipo de acciones se pueden realizar porque somos conscientes de que la pizza es un **valor futuro**, algo que espero obtener y usar más adelante.

Llegado un momento, escucho el número de nuestro pedido y podemos regresar al mostrador con el ticket con nuestro número de pedido y recoger nuestra pizza.

En otras palabras, cuando mi valor futuro está listo, podemos intercambiar nuestra promesa de valor (ticket) por el valor en sí (pizza).

También existe otra posibilidad. Se puede escuchar el número de nuestro pedido, pero el dependiente nos puede informar de que no tienen ingredientes suficientes y no pueden darnos nuestra pizza. A pesar de que no es lo esperado, podemos ver una característica importante del **valor futuro**: puede indicar éxito o fracaso.

Ahora que hemos ejemplificado el comportamiento de una promesa, nos damos cuenta de que podemos representar una promesa como si fuera un tipo básico, ya que podemos unificar su comportamiento en los siguientes estados:

- **Pendiente** (*pending*): estado inicial, no cumplida o rechazada.
- **Cumplida** (*fulfilled*): significa que la operación se completó satisfactoriamente.
- **Rechazada** (*rejected*): significa que la operación falló.

La forma más sencilla de crear una promesa es llamando a `Promise.resolve`. Esta función asegura que el valor que das cumple los requisitos para ser una promesa. Si ya es una promesa, simplemente la retorna (más adelante veremos cómo crear

nuestras propias promesas). En caso de que no lo sea, se obtiene una nueva promesa que termina inmediatamente con el valor como resultado:

```
let numberPi = Promise.resolve(3.14);
numberPi.then(value => console.log `Pi is ${value}`);
// Shows 3.14 immediately, as 3.14 is not a Promise
```

Para obtener el resultado de una promesa se puede usar el método `then`. Este método ejecuta la función `callback` dada como parámetro cuando la promesa se resuelve y produce un valor. Se pueden añadir múltiples *callbacks* a una sola promesa y serán llamadas en orden, incluso si se añaden cuando la promesa ya haya sido resuelta (*fulfilled*).

Para crear una promesa, se puede usar `Promise` como constructor. Este constructor espera una función como argumento, que es llamada inmediatamente, pasándole una función que puede usar para resolver la promesa. En lugar de usar el método `resolve`, esta función será la encargada de resolverla.

En este ejemplo se puede ver cómo se crea una interfaz basada en promesas para la función `readStorage`:

```
function storage(nest, name) {
  return new Promise(resolve => {
    nest.readStorage(name, result => resolve(result));
  });
}

storage(bigOak, "enemies")
  .then(value => console.log("Got", value));
```

La función `storage` recibe dos parámetros que son utilizados en la creación de la promesa (`new Promise...`). Esta promesa se encarga de leer el `storage` y una vez accedido al mismo devolver su valor. Para terminar se llama a la función `storage`, y cuando devuelve un valor (espera hasta que la función se resuelva), lo muestra por consola. En este caso se ha utilizado una función flecha para la creación de la función. Su equivalente sin funciones flecha sería el siguiente:

```
function storage(nest, name) {
  return new Promise(function(resolve) {
    nest.readStorage(name, function(resolve) {
      return resolve(result);
    })
  })
}

storage(bigOak, "enemies")
  .then(value => console.log("Got", value));
```

Habitualmente, los cálculos normales en JavaScript pueden fallar, lanzando una excepción. Este procedimiento puede ser muy útil (y real) cuando se trabaja con asincronía, ya que las peticiones de red pueden fallar o parte del código de un método asíncrono puede lanzar una excepción.

Uno de los problemas más comunes del desarrollo con asincronía mediante *callbacks* es que es muy difícil estar seguro de que los errores se reportan correctamente a los *callbacks*.

Como hemos visto en el apartado de *callbacks*, la convención es usar el primer argumento para indicar que la acción ha fallado y el segundo contiene el valor devuelto por la acción cuando todo va bien. Estas funciones deben asegurarse si han recibido una excepción y se gestiona correctamente.

Con promesas, resolver este tipo de problemas es mucho más sencillo. En función del estado asignado a dicha promesa, se puede desencadenar una acción u otra. Por ejemplo, si una promesa cambia su estado a *fulfilled*, se pueden desencadenar acciones posteriores (mediante `.then()`, por ejemplo) y continuar con la ejecución del programa. En caso de que el estado cambie a *rejected*, se puede continuar con la gestión de errores de dicha llamada (mediante `.catch()`).

Veámoslo con un sencillo ejemplo:

```
// url (required), options (optional)
fetch('https://url.com/endpoint/', {
  method: 'get'
}).then(function(response) {
  // Wherever
}).catch(function(err) {
  // Error :(
});
```

La gestión de la asincronía se realiza de un modo conceptualmente muy sencillo:

- al ser ejecutada la primera línea, `fetch` crea una promesa en estado pendiente;
- cuando se resuelva esta, podrá pasar a un estado completado o rechazado;
- si se rechaza, se ejecutará el método `catch` con la función definida como `callback`, que recibirá como parámetro el error (err) encontrado;
- si se completa, se ejecutará el método `then` con la función definida como `callback`, que recibirá como parámetro la respuesta (response) de la petición.

3. Promesas

3.3. Then()

Previamente, hemos descrito qué es una promesa, su estructura y cómo se comporta, pero no hemos hablado de cómo trabajar con ellas o qué hacer con ellas.

El trabajo de resolución de una promesa hace la asincronía más sencilla, ya que los posibles estados se resuelven dentro de `then()` cuando se ha resuelto correctamente, y lanza una excepción en caso de que haya habido algún error:

```
new Promise((resolve, reject) => {...})
  .then(value => console.log("This happens after the promise is
resolved"))
  .catch(err => {
    console.log("Error: " + err);
    return;
  });
```

Cuando trabajamos con promesas es importante saber si un valor es una promesa o no, o si es un valor que se comporta como tal. El modo más sencillo de saberlo es que tenga implementado el método `then()` (es una función `thenable`).

3. Promesas

3.4. La API de *promise*

El objeto *promise* cuenta con una serie de métodos estáticos que se pueden utilizar cuando se operan con ellas. A continuación, vamos a hacer un pequeño repaso de las más importantes.

Enlace recomendado

En la web de MDN se pueden consultar detalladamente todos los métodos y su uso:

https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Global_Objects/Promise#m%C3%A9todos_est%C3%A1ticos.

- **Promise.all()**. Este método espera a que se resuelvan todas las promesas o se rechace alguna de un iterable (normalmente un `array` de promesas) y devuelve una nueva promesa.

```
Promise.all(iterable).then((res) => {  
    // Here we can work with the promise response  
})
```

- **race()**. Este método es similar a `Promise.all()`, pero solo espera a la primera promesa que se resuelva. Devuelve una nueva promesa con la promesa resuelta.

```
Promise.race([  
    new Promise((resolve, reject) => setTimeout(() => resolve(1), 5000)),  
    new Promise((resolve, reject) => setTimeout(() => reject(new  
Error("Rejected promise"), 2000)),  
    new Promise((resolve, reject) => setTimeout(() => resolve(3), 1000))  
]).then(alert); // 3
```

- **resolve()** y **Promise.reject()**. El método `Promise.resolve(value)` retorna un objeto `Promise` que es resuelto con el valor dado. Asimismo, el método `Promise.reject(reason)` retorna un objeto `Promise` que es rechazado por la razón que se especifique.

Estos métodos se usan raramente en código moderno, porque la sintaxis de `async/await` que veremos posteriormente las hace menos útiles, aunque puede ser un recurso útil en la depuración de código complejo.

```
let promise = new Promise(resolve => resolve(value));  
let promise = new Promise((resolve, reject) => reject(error));
```

3. Promesas

3.5. Ejercicios

Ejercicio 13

Modifica la siguiente función para que siga las siguientes reglas:

- Retornar una promesa.
- Resolver esta promesa tras 5 segundos con el valor de `piNumber` .

```
function asyncPi() {  
  const pi = 3.14;  
}
```

Ejercicio 14

Modifica la siguiente función para que siga las siguientes reglas:

- Devolver una promesa.
- Si el parámetro de entrada no es un número, debería rechazarse después de 1 segundo con el siguiente texto como datos: *Not a Number*.
- Si el parámetro de entrada es un número, debería resolver la promesa después de 10 segundos con la siguiente información: *(número de data) is a number*.

```
function isANumber(data) {  
}
```

Ejercicio 15

Modifica el siguiente código para que siga las siguientes reglas:

- Llamar al método `asyncCall` .
- Cuando el método termine y devuelva información, mostrar esa información por consola.

```
function asyncCall() {  
  return new Promise(resolve => {  
    setTimeout(() => {  
      resolve("You should see this message in console after 5  
secs");  
    }, 5000);  
  })  
}
```

Ejercicio 16

Analiza el siguiente código. ¿Cuál sería su salida por consola y por qué?

```
function asyncFunction() {  
  return new Promise(function(resolve, reject) {  
    reject();  
  });  
}
```

```

let promise = job();

promise
  .then(function() {
    console.log('First then');
  })
  .then(function() {
    console.log('Second then');
  })
  .then(function() {
    console.log('Third then');
  })
  .catch(function() {
    console.log('First error');
  })
  .then(function() {
    console.log('Fourth then');
  });

```

Ejercicio 17

Modifica el siguiente código para que siga las siguientes reglas:

- Generar una promesa que encadene las promesas que devuelvan las siguientes funciones.
- Cuando se resuelvan todas las promesas, mostrar por consola los datos de cada método y, al final, mostrar el texto *Finish*.

```

let firstPromise = new Promise(function(resolve, reject) {
  setTimeout(resolve, 5000, 'firstPromise');
});

let secondPromise = new Promise(function(resolve, reject) {
  setTimeout(resolve, 1000, 'SecondPromise');
});

```

Ejercicio 18

Modifica el siguiente código para que cumpla con los siguientes requisitos:

- Mostrar por pantalla las respuestas de `p1`, `p2` y `p3` una vez se resuelvan
- Mostrar por pantalla el texto *error*, en el caso de que falle alguna llamada.

```

const p1 = new Promise((resolve, reject) => {
  setTimeout(() => {
    console.log('First Promise');
    resolve("First");
  }, 1000);
});

const p2 = new Promise((resolve, reject) => {
  setTimeout(() => {
    console.log('Second Promise');

```

```

        resolve('Second');
    }, 2000);
});
const p3 = new Promise((resolve, reject) => {
    setTimeout(() => {
        console.log('Third promise');
        resolve("Third");
    }, 10000);
});

```

Ejercicio 19

¿Cuál sería la salida por pantalla del código previamente desarrollado si el contenido de `p2` fuera el siguiente? Intenta deducirlo sin tener que ejecutarlo. Describe brevemente su comportamiento.

```

const p2 = new Promise((resolve, reject) => {
    setTimeout(() => {
        console.log('Second Promise');
        reject('Second');
    }, 2000);
});

```

Ejercicio 20

Modifica el siguiente código para que cumpla con los siguientes requisitos:

- Mostrar por pantalla la respuesta de la primera promesa que se resuelva.
- Mostrar por pantalla el texto *error*, en el caso de que falle primero una promesa.

```

const p1 = new Promise((resolve, reject) => {
    setTimeout(() => {
        console.log('First Promise');
        resolve("First");
    }, 1000);
});
const p2 = new Promise((resolve, reject) => {
    setTimeout(() => {
        console.log('Second Promise');
        resolve('Second');
    }, 2000);
});
const p3 = new Promise((resolve, reject) => {
    setTimeout(() => {
        console.log('Third promise');
        reject("Third");
    }, 2000);
});

```

4. Generadores

4.1. Introducción

Anteriormente, hemos hablado tanto de *callbacks* y sus problemas para la gestión de asincronía como de promesas y de cómo solucionan los problemas de inversión de control.

Ahora, vamos a centrarnos en cómo expresar el flujo de control de asincronía de un modo secuencial, para que parezca código síncrono. Esta «magia» la hacen posible los generadores de ES6.

Al definir una función con un asterisco (`function*`), se convierte en un generador. Cuando se realiza una llamada a un generador, el valor devuelto es un iterador.

Nota

ES6 hace referencia a la sexta versión de JavaScript (conocido formalmente como ECMAScript). También puede ser referido como ECMAScript2015 por su año de publicación. Esta versión marcó un antes y después en el lenguaje de programación por la profundidad de sus cambios y abrió la puerta a su uso en

aplicaciones complejas; lo que mejoró sensiblemente el desarrollo de funcionalidades con asincronía, por ejemplo.

```
function* iterator(n) {  
  for (let current = n; current += n) {  
    yield current;  
  }  
}  
  
for (let iteration of iterator(3)) {  
  if (iteration > 10) break;  
  console.log(iteration);  
}  
// → 3  
// → 6  
// → 9
```

Inicialmente, al llamar al método `iterator`, la función se queda congelada. Cada vez que se llama a `next` sobre el iterador (o se recorre el bucle mediante el iterador `for`), se ejecuta el contenido de la función hasta que se alcanza la expresión `yield`, lo que retorna el valor que se desee. En nuestro caso, este valor retornado será la suma del valor previo. La función retorna y finaliza cuando se hace uso de la sentencia `return`, que en el caso de ejemplo no ocurre, por lo que continúa en ejecución hasta que la sentencia de control lanza un `break`.

Hay que hacer mención especial al método `next()`, inherente a las funciones generador. Una vez creado un iterador, como en el ejemplo anterior, se puede recorrer mediante el método `next()`. Este método retorna un objeto con las siguientes propiedades:

- **Value**: el siguiente valor en la secuencia de la función generador.
- **Done**: booleano, que es `true` si el valor ya se ha consumido previamente. Se suele asociar a que ya se han terminado de recorrer todos los valores posibles del iterador (como si fuera un *array*).

4. Generadores

4.2. Generadores y *promises*

Una vez comprendido el funcionamiento de los generadores es fácil darse cuenta de que puede ser un buen método para generar un patrón de desarrollo y gestionar asincronía con promesas. Digamos que tenemos una función generador que produce instancias de promesas.

```
const generator = function* generator() {  
  const a = yield Promise.resolve(1);  
  const b = yield Promise.resolve(2);  
  return a + b;  
};  
  
const iterator = generator();  
iterator.next();  
iterator.next();  
iterator.next();
```

Este código retorna con cada llamada a `next()` los siguientes valores:

```
{ value: Promise(1), done: false }  
{ value: Promise(2), done: false }  
{ value: NaN, done: true }
```

Como se puede ver, el generador trata las promesas igual que cualquier otro valor, sin ofrecer ninguna ventaja ni ayuda para gestionar llamadas asíncronas.

Pero es posible crear una función llamada `async`, que acepte cualquier función generadora y cree una promesa resolviendo el contenido de nuestro generador correctamente.

```
const promisory = async (function* generator() {  
  const a = yield Promise.resolve(1);  
  const b = yield Promise.resolve(2);  
  
  return a + b;  
})();  
  
promisory();
```

Esta función retorna lo siguiente:

```
Promise(3);
```

Con este patrón se pueden generar programas más complejos. Los métodos generadores de promesas son funciones compuestas de lenguaje imperativo, lo que facilita enormemente la lectura en comparación con la programación con *callbacks*. Además, es más seguro y predecible, ya que se elimina también la inversión de control. Para terminar, es importante indicar que este tipo de patrón se acerca a los principios de programación funcional.

4. Generadores

4.3. Async/Await

Como evolución al desarrollo con promesas y tratando de facilitar el trabajo con las mismas, posteriormente se añadió a la definición de JavaScript, las funciones *async*, un tipo de funciones específico para definir funciones asíncronas, usando como referencia la estructura de funcionamiento con generadores. Esto ayuda a poder leer el código de un modo lineal como si de código síncrono se tratase. La sintaxis de este tipo de función es similar a nuestro desarrollo previo de funciones asíncronas con generadores:

```
async function name(param0, param1, /* ... ,*/ paramN) {  
  statements  
}
```

Este tipo de funciones *async* retornan una promesa que será resuelta con el valor del código que contiene o rechazada lanzando una excepción. Por tanto, las siguientes dos funciones serían similares:

```
async function hey() {  
  return 'Ya!';  
}
```

```
function hey() {  
  return Promise.resolve('Ya!');  
}
```

Las funciones asíncronas pueden contener la expresión `await` o no, y a su vez lo puede contener un número indefinido de veces.

Este operador, exclusivo de las funciones *async* permite pausar la ejecución del código de la función hasta que la promesa a la que se asigna termine o sea rechazada.

```
async function asyncMethod() {  
  const res = await resolveAPromise();  
  return res;  
}  
  
const asyncMethod = async () => {  
  const res = await resolveAPromise();  
  return res;  
}
```

Otra ventaja de las funciones *async* es que el uso de funciones *async* con `await` permite el uso convencional de los bloques `try/catch` alrededor de código asíncrono.

```
async function randomFunction(param) {  
  try {  
    res = await randomCall();  
  }  
  catch (e) {  
    randomManageErrorFunction(e);  
  }  
  return res.data;  
}
```


4. Generadores

4.4. Ejercicios

Ejercicio 21

Escribe una función generador `splitSentence` que cumpla los siguientes requisitos:

- Recibir como parámetro una frase.
- Separar esta frase por palabras.
- Cada vez que se llama a `next()` devolver una palabra hasta que termine.

Ejercicio 22

Escribe una función generador de números de Fibonacci con los siguientes requisitos:

- Tener como nombre *limitedFibonacci*.
- Recibir como parámetro un número *n* que definirá el máximo de números que puede devolver.
- Calcular la secuencia.
- Cuando se alcance el máximo *n* de llamadas, la función generador deberá retornar como *done=true*.

Ejercicio 23

Reescribe la siguiente función como una función `async` :

```
function processRandomData(url) {  
  return getRandomDataFromUrl(url)  
    .catch((e) => sendErrorMessage(url))  
    .then((v) => doSomethingCoolWithData(v));  
}
```

Ejercicio 24

Reescribe la siguiente función como una función `async` teniendo en cuenta las siguientes consideraciones:

- El parámetro `result` es una promesa que lanza una id cuando se resuelve.

```
function job(result, database, errorManager) {  
  return result  
    .then(function(id) {  
      return database.get(id);  
    })  
    .then(function(info) {  
      return info.name;  
    })  
    .catch(function(error) {  
      errorManager.notify(error);  
      throw error;  
    });  
}
```

Ejercicio 25

Modifica el siguiente código para que sea gestionado mediante `async/await` , teniendo en cuenta las siguientes consideraciones:

- El método `randomAfter5Seconds` devuelve una promesa que se resuelve cada 5 segundos con un valor numérico al azar.

```
function randomMathPromise(x) {
  return new Promise(resolve => {
    randomAfter5Seconds().then((a) => {
      randomAfter5Seconds().then((b) => {
        randomAfter5Seconds().then((c) => {
          resolve(x + a - b * c);
        })
      })
    })
  });
}

randomMathPromise(7).then((calc) => {
  console.log(calc);
});
```

Ejercicio 26

Reescribe el siguiente código utilizando `async/await` en lugar de `then/catch` :

```
function loadJsonFromAPI(url) {
  return fetch(url)
    .then(response => {
      if (response.status == 200) {
        return response.json();
      } else {
        throw new Error(response.status);
      }
    });
}

loadJsonFromAPI('https://randomservice.com/apiendpoint')
  .catch(alert);
```

5. Soluciones de los ejercicios

Ejercicio 1

```
Main Block
Timeout
```

No muestra el bloque de la función, ya que no se está instanciando, por lo que primero se muestra el mensaje `Main block`, ya que no tiene `timeout` y termina con el bloque de `timeout`.

Ejercicio 2

```
(function randomFunction() { console.log("Function"); })()
randomFunction()
```

Para instanciar una función se puede hacer inmediatamente en la propia declaración de la misma. Este tipo de funciones son conocidas como IIFE (*immediately invoked function expression*).

Ejercicio 3

```
const first = () => console.log('First');
const second = () => setTimeout(() => console.log('Second'));
const third = () => console.log('Third');
```

El hecho de utilizar funciones flecha no modifica el comportamiento de ejecución de las siguientes funciones. Simplemente se ejecutan en la misma línea. Por tanto, la salida de consola sería:

```
First
Third
Second
```

La función `setTimeout` se encola y se ejecuta cuando termine la ejecución de las funciones existentes. En este caso, las que contienen el `console.log()` de `first` y `third`.

Ejercicio 4

```
for ( var i = 0; i < 3; i++) {
  setTimeout( function() { console.log(i); }, i * 1000 );
}
```

Mostraría por pantalla tres veces 3, ya que el bloque del bucle `for` se ejecuta y completa antes de los tres bloques `setTimeout`, que se añaden posteriormente a la pila de llamadas.

Ejercicio 5

```
function first() {
  console.log(1)
}

function second() {
  setTimeout(() => {
    console.log(2)
  }, 0)
}

function third() {
```

```

    console.log(3)
  }

  first()
  second()
  third()

```

Mostraría por pantalla 1, 3, 2. Primero se resolvería `first()` y `third()`, ya que dentro de estos métodos no hay otro bloque que meter en la pila de llamadas. Para terminar, se resolvería el bloque que contiene el `setTimeout`, que al tener 0 como retraso se mostraría sin espera.

Ejercicio 6

```

( function() {
  console.log(10);
  setTimeout( function() { console.log(20) }, 1000 );
  setTimeout( function() { console.log(30) }, 0 );
  console.log(40);
} )();

```

Mostraría primero 10 y 40, ya que son las primeras sentencias que se ejecutan dentro del bloque de la función. Posteriormente, resolvería los dos bloques de `setTimeout`, pero mostraría 30 primero, puesto que tiene un retraso de 0 milisegundos.

Ejercicio 7

```

function greet(name, callback) {
  console.log('Hello Mr.' + ' ' + name);
  callback();
}

function farewell() {
  console.log('Goodbye Mr. Jose');
}

greet('Pepito', farewell);

```

Ejercicio 8

```

function greet(name) {
  console.log('Hello Mr.' + ' ' + name);
  callback(name);
}

function farewell(name) {
  console.log('Goodbye Mr.' + ' ' + name);
}

setTimeout(greet, 3000, 'Pepito', farewell);

```

Ejercicio 9

```

function builtCar(name) {
  setTimeout(() => {
    console.log('Building a' + ' ' + name);
  }, 1000);
}

```

```

    callback();
}

function defineCar(name) {
    console.log('This car is a ' + ' ' + name);
}

builtCar("Ford Focus", defineCar);

```

Ejercicio 10

```

function hellishFunction() {
    setTimeout(() => {
        console.log(1)
        setTimeout(() => {
            console.log(2)
            setTimeout(() => {
                console.log(3)
            }, 500)
        }, 2000)
    }, 1000)
}

```

Este ejercicio es similar a los del apartado 1, ya que hay que tener en cuenta el *event loop*, pero ejemplifica de un modo más gráfico el comportamiento y la complejidad de resolver código con *callback hell*. La salida por pantalla de las funciones previas sería 1, 2, 3, a pesar de que los *timeouts* son menores en 3 y 2.

Ejercicio 11

```

const visited = []

function walk() {

    setTimeout(function() {
        visited.push('London');

        setTimeout(function() {
            visited.push('Milan');

            setTimeout(function() {
                visited.push('Madrid');

                setTimeout(function() {
                    visited.push('Barcelona');
                    console.log(visited.toString())
                }, 1000);
            }, 1000);
        }, 1000);
    }, 1000);
}

walk()

```


La salida por consola sería London, Milán, Madrid, Barcelona. La razón es similar a la del ejercicio previo, solo que en este caso al tener un `timeout` similar, las peticiones se van encolando y desencolando en el mismo orden.

Ejercicio 12

```
const mainFunction = (callback) => {
  setTimeout(() => {
    callback([10, 3, 7]);
  }, 3000)
}

const addFunction = (array) => {
  let sum = 0;
  for(let i of array) {
    sum += i;
  }
  console.log(sum);
}

mainFunction(add);
```

La salida por consola será 20. En este caso, simplemente se esperan 3 segundos a llamar a la función `addFunction`, y posteriormente se suman los elementos del `array` y se muestran por consola.

Ejercicio 13

```
function asyncPi() {
  const pi = 3.14;
  return new Promise(resolve => {
    setTimeout(() => {
      resolve(pi);
    }, 5000);
  })
}
```

Ejercicio 14

```
function isANumber(data) {
  return new Promise((resolve, reject) => {
    if(isNaN(data)) {
      setTimeout(() => {
        reject("Not a number")
      }, 1000);
    }
    else {
      setTimeout(() => {
        resolve(`${data} is a number`)
      }, 11000)
    }
  });
}
```

Ejercicio 15

```
function asyncCall() {
  return new Promise(resolve => {
    setTimeout(() => {
      resolve("You should see this message in console after 5
secs");
    }, 5000);
  })
}

asyncCall().then(res => console.log(res));
```

Ejercicio 16

```
// First error
// Fourth then
```

Ejercicio 17

```
let firstPromise = new Promise(function(resolve, reject) {
  setTimeout(resolve, 5000, 'firstPromise');
});

let secondPromise = new Promise(function(resolve, reject) {
  setTimeout(resolve, 1000, 'SecondPromise');
});

let promise = Promise.all([firstPromise, secondPromise]);
promise.then(function(data) {
  data.forEach(function(data) {
    console.log(data);
  });
  console.log("Finished");
})
```

Ejercicio 18

```
const p1 = new Promise((resolve, reject) => {
  setTimeout(() => {
    console.log('First Promise');
    resolve("First");
  }, 1000);
});

const p2 = new Promise((resolve, reject) => {
  setTimeout(() => {
    console.log('Second Promise');
    resolve('Second');
  }, 2000);
});

const p3 = new Promise((resolve, reject) => {
  setTimeout(() => {
    console.log('Third promise');
  }, 3000);
});
```

```

        resolve("Third");
    }, 10000);
});

Promise.all([p1, p2, p3])
    .then((res) => console.log(res))
    .catch("Error");

```

La realización de acciones una vez se resuelvan todas las promesas se gestiona mediante `Promise.all`. Recibirá como parámetros un array con todas las promesas y devolverá una promesa en la que se podrá utilizar tanto `then()` como `catch()`.

Ejercicio 19

El código resultante sería el siguiente:

```

const p1 = new Promise((resolve, reject) => {
    setTimeout(() => {
        console.log('First Promise');
        resolve("First");
    }, 1000);
});

const p2 = new Promise((resolve, reject) => {
    setTimeout(() => {
        console.log('Second Promise');
        reject('Second');
    }, 2000);
});

const p3 = new Promise((resolve, reject) => {
    setTimeout(() => {
        console.log('Third promise');
        resolve("Third");
    }, 10000);
});

Promise.all([p1, p2, p3])
    .then((res) => console.log(res))
    .catch("Error");

```

En este caso al contar con un `reject()` nunca pasaría por la función `then()`, y mostraría solo *error*, debido a que `Promise.all` solo resuelve una promesa exitosa si todas las promesas hacen `resolve`.

Ejercicio 20

Este es un caso similar al ejercicio 18, pero en lugar de utilizar `Promise.all`, se hace uso de `Promise.race`. El código resultante sería el siguiente:

```

const p1 = new Promise((resolve, reject) => {
    setTimeout(() => {
        console.log('First Promise');
        resolve("First");
    }, 1000);
});

```

```

    }, 1000);

});
const p2 = new Promise((resolve, reject) => {
  setTimeout(() => {
    console.log('Second Promise');
    resolve('Second');
  }, 2000);
});
const p3 = new Promise((resolve, reject) => {
  setTimeout(() => {
    console.log('Third promise');
    reject("Third");
  }, 2000);
});

Promise.race([p1, p2, p3])
  .then((res) => console.log(res))
  .catch("Error");

```

Ejercicio 21

```

function* splitSentence(sentence) {
  const words = sentence.split(' ');
  for (let i = 0; i < words.length; i++) {
    yield words[i];
  }
}

```

Ejercicio 22

```

function* limitedFibonacci(n) {
  let [a, b] = [0, 1];
  while (a <= n) {
    yield a;
    [a, b] = [b, a + b];
  }
}

```

Ejercicio 23

```

async function processRandomData(url) {
  let v;
  try {
    v = await getRandomDataFromUrl(url);
  } catch (e) {
    v = await sendErrorMessage(url);
  }
  return doSomethingCoolWithData(v);
}

```

Ejercicio 24

```
async function job(result, database, errorManager) {
  let id;
  let info;
  try {
    id = await result;
    info = await database.get(id)
  }
  catch (error) {
    errorManager.notify(error);
    throw error;
  }
  return info.name;
}
```

Ejercicio 25

```
async function randomMathPromise(x) {
  const a = await randomAfter5Seconds();
  const b = await randomAfter5Seconds();
  const c = await randomAfter5Seconds();
  return x + a - b * c;
}

randomMathPromise(10).then((sum) => {
  console.log(sum);
});
```

Ejercicio 26

```
async function loadJsonFromAPI(url) {
  let response = await fetch(url);

  if (response.status == 200) {
    let json = await response.json();
    return json;
  }

  throw new Error(response.status);
}

loadJsonFromAPI('https://randomservice.com/apiendpoint')
  .catch(alert);
```

Bibliografía

Haverbeke (2014). Eloquent JavaScript: A Modern Introduction to Programming (2.ª edición). No Starch Press, Incorporated.
<https://discovery.biblioteca.uoc.edu/permalink/34CSUC_UOC/166h2gj/cdi_skillsoft_books24x7_bks00077614>

JavaScript | MDN (2022, 4 de agosto). Retrieved September 26, 2022, from
<https://developer.mozilla.org/es/docs/Web/JavaScript>. <<https://developer.mozilla.org/es/docs/Web/JavaScript>>

Simpson (2016). You don't know JS#: ES6 and beyond / Kyle Simpson. (1.ª edición). O'Reilly.
<https://discovery.biblioteca.uoc.edu/discovery/fulldisplay?context=L&vid=34CSUC_UOC:VU1&search_scope=MyInst_and_CI&tab=Everything&docid=alma991000733936906712/alma991000733936906712>

Simpson (2015). Async & performance / Kyle Simpson. (1.ª edición). O'Reilly.
<https://discovery.biblioteca.uoc.edu/discovery/fulldisplay?context=L&vid=34CSUC_UOC:VU1&search_scope=MyInst_and_CI&tab=Everything&docid=alma991000733575006712>